

LLM Model Evaluations: Core Capabilities and Benchmarking

Why AI Engineers Need Model Evals

We are studying this course specifically from the perspective of AI Engineers whose day-to-day job is building LLM-based applications. While frontier AI labs (like OpenAI or Anthropic) require model evaluations to understand training performance and shape their models, we need them for our own practical application development.

As the saying goes, "**If you can't measure, you can't improve.**" Without model evaluations, we are essentially blind in our development process. Specifically, as AI Engineers, we need model evaluations for four key reasons:

1. **Model Selection & Comparison:** When designing an application, you must decide which LLM will act as the "brain" of your system. For example, should you choose OpenAI or Claude? In a professional team meeting, you cannot simply state that "both are good, choose whichever". You must present concrete, data-backed pointers showing which model scores higher on the specific capabilities required by your application.
 2. **Tracking Improvements in New Models:** If your application is deployed using an older model like Claude Opus and a new model like Claude Fable is released, you need a reliable way to determine if you should upgrade. Model evaluations provide the exact numbers and metrics required to justify whether the new model is actually an improvement.
 3. **Assessing Safety and Reliability:** Model evaluations tell us how safe a model is, how much it hallucinates, and how vulnerable it is to adversarial jailbreaks.
 4. **Proprietary vs. Open-Source Decisions:** You must decide whether to use a proprietary API (like Claude) or host an open-source model (like DeepSeek) on your company's servers. While open-source might be cheaper, proprietary models might perform better across key capabilities. Model evaluations facilitate this critical cost-benefit comparison.
-

What is Model Evaluation?

Formally, **Model Evaluation is a systematic process of measuring an underlying model's capabilities, behavior, reliability, and operational characteristics under controlled conditions.**

The 4-Step Technical Protocol of Model Evals:

Every model evaluation follows a standardized, four-step process:

1. **Decide the Capability to Test:** LLMs are general-purpose models with a vast array of skills.

Unlike humans, where a single IQ score can represent a lot of information, LLMs do not have a single evaluation metric that represents their overall capability. We must first isolate the specific capability we want to test (e.g., reasoning, coding, safety, or instruction following).

2. **Bring a Test / Evaluation Mechanism:** We bring a specific test or mechanism to measure that capability. This test typically falls into one of two categories:
 - **Benchmarks:** Standardized, public, and shared tests (e.g., **MMLU** or **SWE-bench**). Because every model runs the exact same test, benchmarks are excellent for comparing different models on common ground.
 - **Custom Evaluation Sets:** Datasets assembled from actual tasks to measure the specific requirements we care about for our application, rather than generic capabilities.
 3. **Run the Model Under a Fixed Protocol:** We run the model through the test in a controlled environment. We fix variables like prompts and environmental conditions to ensure the test is fair, repeatable, and standardized across multiple models.
 4. **Score and Interpret:** Once the test is complete, we calculate the scores and interpret or publish the results.
-

Case Study: Standardized Benchmarks vs. Custom Evaluations

To understand why we cannot rely solely on public benchmarks, let us look at a real-world scenario. Imagine we are building an **Email Routing System for Zomato** to classify incoming emails into three categories: **Billing**, **Technical**, and **Refund**.

We must choose between two models:

- **Model A (The Benchmark Leader):** A large, top-of-the-leaderboard model (similar to Claude Opus). It is highly capable but expensive, costing **\$15 per 1 million tokens**.
- **Model B (The Economical Competitor):** A smaller, mid-table model (similar to a lower-parameter Qwen model). It is highly economical, costing only **50 cents (\$0.50) per 1 million tokens**.

The Benchmarking Trap

If we rely purely on public benchmarks, Model A beats Model B in every single category (math, coding, reasoning, language generation, etc.). Based on benchmarks alone, the obvious choice is Model A. However, deploying Model A at Zomato's massive scale would result in extremely high costs.

The Custom Eval Solution (Building a Golden Data Set)

To make an informed, data-driven decision, we assemble a **Golden Data Set** consisting of **200 to 500 past emails**, manually labeled with their correct categories (Technical, Billing, or Refund). We run both models on this dataset and compile the following operational metrics:

Operational Metric	Model A (Big & Expensive)	Model B (Small & Cheap)
Classification Accuracy	94%	91%
Urgency Accuracy (Detecting reply urgency)	88%	87%
Cost (per 1,000 emails)	\$6.00	Less than \$0.21
Latency (Time to serve request)	4.1 seconds	Just 9 seconds

Key Takeaway

Based on this custom evaluation, Model B is the clear winner for our specific task. We lose only a marginal amount of accuracy (3% in classification and 1% in urgency) but we save massive amounts of money and achieve practical latency for our users.

If we had only relied on public benchmarks, we would have concluded that Model A is the only viable option because it beat Model B on every benchmark. However, running a custom evaluation on our own data revealed that Model B is a much better value proposition for our specific application.

Therefore:

- **Benchmarks** provide generic capabilities and common-ground comparisons.
- **Custom Evals** tell us which model is truly suitable for our specific business application and data.

The 8 Core LLM Capabilities Tested by Benchmarks

Most of the public benchmarks you will encounter in the future are designed to target one or more of these **eight core capabilities** that define an LLM's performance:

1. Knowledge and Reasoning

These two are clubbed together because they are deeply interconnected in real-world use.

- **Factual Recall:** This measures how much factual knowledge the model retained from its training data across diverse fields of study. For example, the benchmark **MMLU** tests models on **57 distinct subjects** (including Biology, Physics, Chemistry, History, etc.).
- **Multi-Step Logical Reasoning:** This measures the model's ability to connect distinct facts in a logical sequence to arrive at a correct conclusion.
 - *Example task:* Asking the model to summarize human evolution starting from the Big Bang all the way to modern times, and then analyzing why today's society is structured the way it is. This tests both factual recall of historical events and the reasoning ability to connect those events to modern societal structures.
- **Why Frontier Labs Care:** This is the ultimate metric for "intelligence". A model's score on

reasoning benchmarks is how the industry judges its overall cognitive power, and every model provider wants to score high here.

- **Real-World Applications:** Building research chatbots that analyze academic papers to discover new insights, parsing highly complex and technically accurate documents (like Machine Learning papers), or creating specialized assistants that help professionals (such as lawyers or teachers) in their fields.

2. Coding and Software Engineering

This capability is one of the most economically valuable features of LLMs today (as demonstrated by specialized tools like Cursor reaching valuations of \$60 billion). It measures whether a model can generate code that actually runs and performs complex software engineering tasks.

- **Functional-Level Code Generation:** Generating a working function (e.g., in Python) based on a natural language prompt, generating appropriate test cases, and iteratively improving the code based on errors caught by those test cases.
- **Bug Fixing:** Going into an existing codebase, identifying bugs, and fixing them.
- **Multi-File Long-Horizon Engineering:** Handing the model an entire multi-file codebase and asking it to refactor a specific aspect across the whole project.
- **Command Line Operations:** Setting up server environments, installing software packages, and configuring systems directly via terminal commands.
- **API and Function Calling:** Correctly identifying when to call an API and structuring the payload.
- **Real-World Applications:** Powering fully autonomous AI Coding Agents.

3. Mathematics

Mathematics is treated as a highly specific and rigorous form of step-by-step logical reasoning, but with deep, specialized applications.

- **Grade School Level:** Basic mathematics (7th and 8th-grade level algebra and arithmetic).
- **Competition Level:** Highly creative problem-solving required in math Olympiads.
- **Undergraduate & Research Level:** Solving advanced college-level problems or assisting with open-ended, unsolved mathematical research where solutions do not yet exist.
- **Real-World Applications:** Engineering simulations, scientific computing, financial modeling, and advanced data analysis.

4. Long Context

While modern models boast massive **context windows** (ranging from 128k to 200k, and even up to 1 million tokens), their actual ability to use that context effectively is what we measure here.

- **The Context Retention Problem:** As a chat session grows longer, a model's ability to retain and retrieve older context diminishes, leading to a degradation in chat quality over time.
- **Specific Tasks Measured:**
 - Extracting a tiny, specific fact hidden deep within a massive document (often called the "needle in a haystack" test).
 - Summarizing extremely long books or documents.
 - Maintaining full context of a large codebase for coding agents.
- **Real-World Applications:** Crucial for any RAG-based application or agent where long documents are fed into the prompt.

5. Vision and Multi-Modality

We live in a multi-modal world, and text alone is no longer sufficient. This capability measures how well models understand inputs beyond text, specifically images and videos.

- **Real-World Applications:** Uploading a photo of your refrigerator's contents and asking the model what dishes you can cook, or using a visual feed in a library to find a specific book on a shelf.

6. Agentic and Tool Use

This capability represents the shift from passive text generation to active execution. The entire field of **Agentic AI** emerged from this core capability.

- **Core Skills Tested:**
 - Determining when and how to use external tools.
 - Web browsing to fetch real-time information.
 - Performing structured tool calling and API interactions.
 - Controlling desktop interfaces and computers to perform tasks.
- **Real-World Applications:** Autonomous agents that can execute tasks in the real world on behalf of the user.

7. Safety and Alignment

This measures how much we can trust the model to behave responsibly, legally, and ethically.

- **Core Areas Tested:**
 - **Harmful Content Prevention:** Ensuring the model refuses to generate illegal, dangerous, or toxic content.
 - **Robustness to Adversarial Attacks:** Measuring how easily the model can be "jailbroken" to bypass its safety guardrails.
 - **Truthfulness vs. Sycophancy (Flattery):** Ensuring the model remains factual and

objective rather than simply agreeing with whatever the user says. *(For instance, in my personal experience, I have always observed that Claude is much more truthful and willing to point out flaws in my ideas, whereas ChatGPT often flatters me and says my idea is the best in the world. We need models to be truthful, not sycophantic.)*

- **Cyber Security Skills:** Testing the model's advanced capabilities in cryptography, reverse engineering, and digital forensics. *(For example, the Claude Fable model is exceptionally strong at identifying and patching software vulnerabilities.)*
- **Why Frontier Labs Care:** Beyond strict government pressure, a minor safety incident represents an immense reputational risk that can completely shut down a frontier lab's business.

8. Instruction Following

An under-rated but highly critical capability that directly impacts user satisfaction. It measures how precisely a model follows specific styling, length, and format constraints.

- **Core Skills Tested:**
 - Adhering to formatting requests (e.g., "answer in a bulleted list").
 - Obeying length limits (e.g., "keep your response under 200 words").
 - Adopting a requested persona or tone (e.g., "answer in a friendly tone").
 - Asking clarifying questions when the user's instructions are ambiguous or unclear.
- **Real-World Importance:** If a model fails to follow basic structural instructions, the user experience suffers, leading directly to user churn to competing platforms.